COSC2633/2637 Big Data Processing

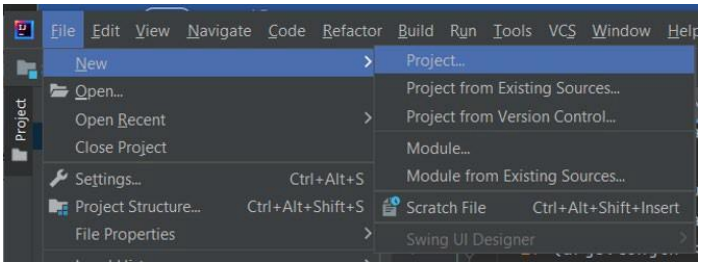
Week 11 - Lab Task

Scala Maven Project on Intellij IDEA

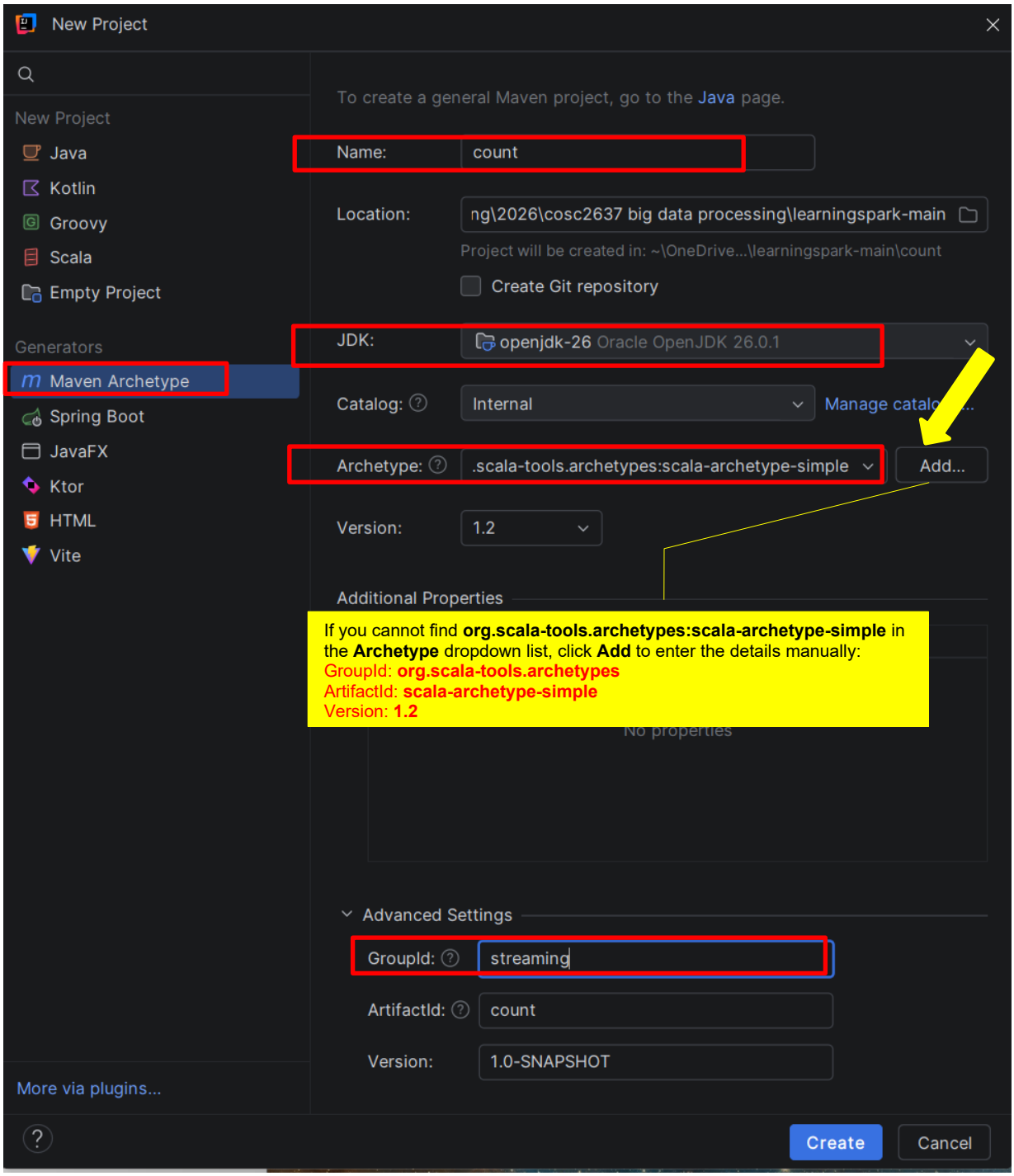
Install IntelliJ IDEA and install the required plugin **scala** into our IntelliJ; after installation, you need restart Intellij IDEA (details can be found in Week 9 – Lab Task).

Task: monitoring an HDFS folder

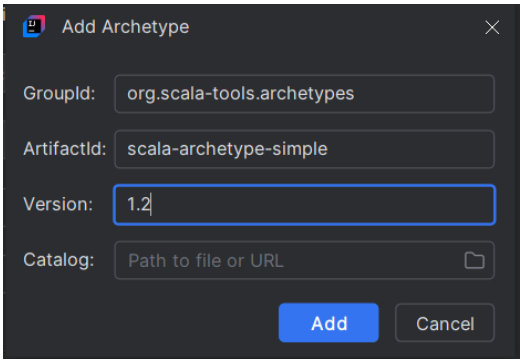
- Create Scala Maven Project



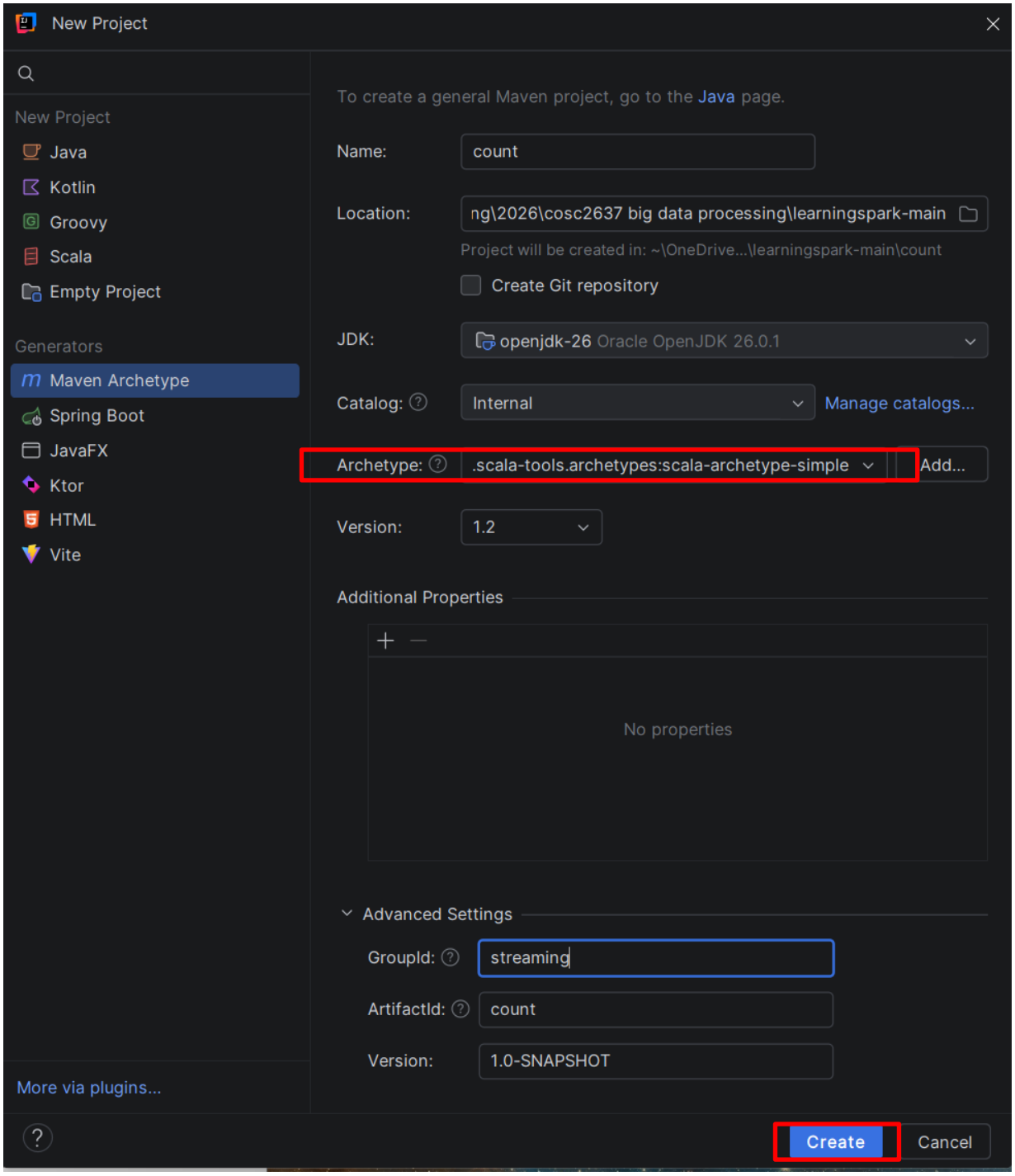
- In **New Project** window, select **Maven Archetype,** name the project (e.g., **count** - if the name has been used previously, just use another name like “mycount”),
- select **org.scala-tools.archetypes:scala-archetype-simple** for **Archetype** (if not in the drop list, you need click **Add** to specify manually).
- In the **Advanced Settings**, name **GroupId** as **streaming**.



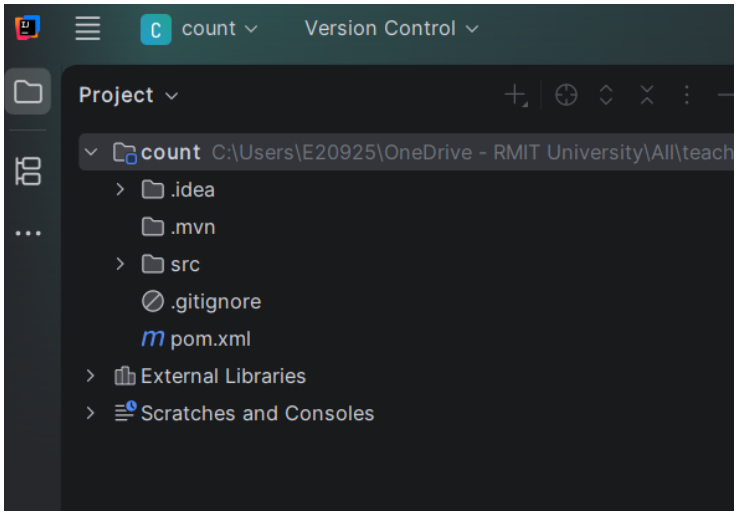
After clicking **Add** for **Archetype,** manually fill **GroupId**, **ArtifactId**, **Version** like below, and then click **Add**



You will see the **Archetype**, Click **Create**,

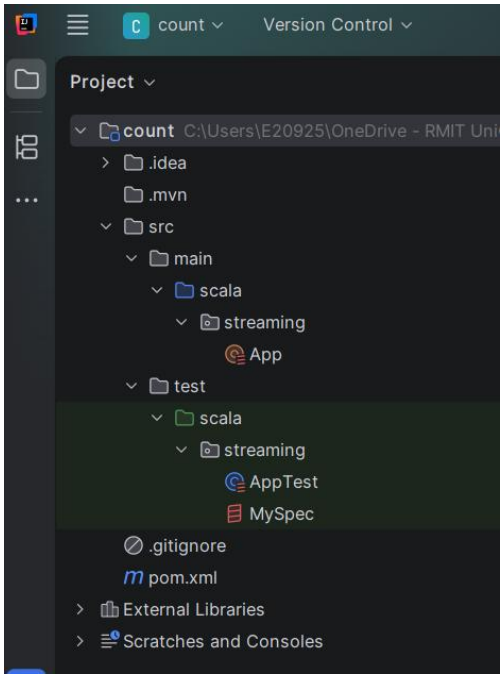


you will see project **count**

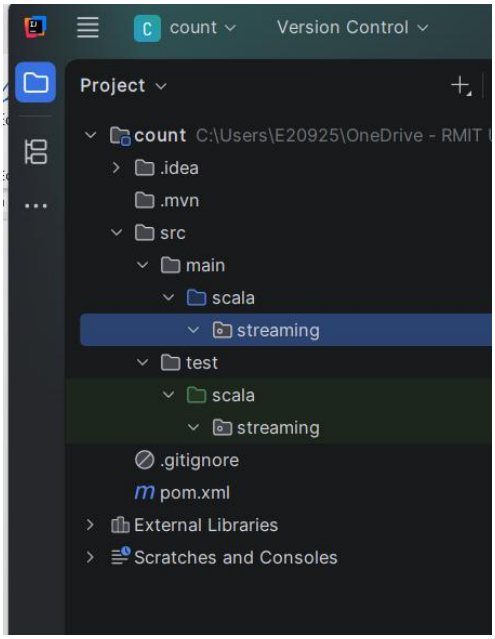


delete

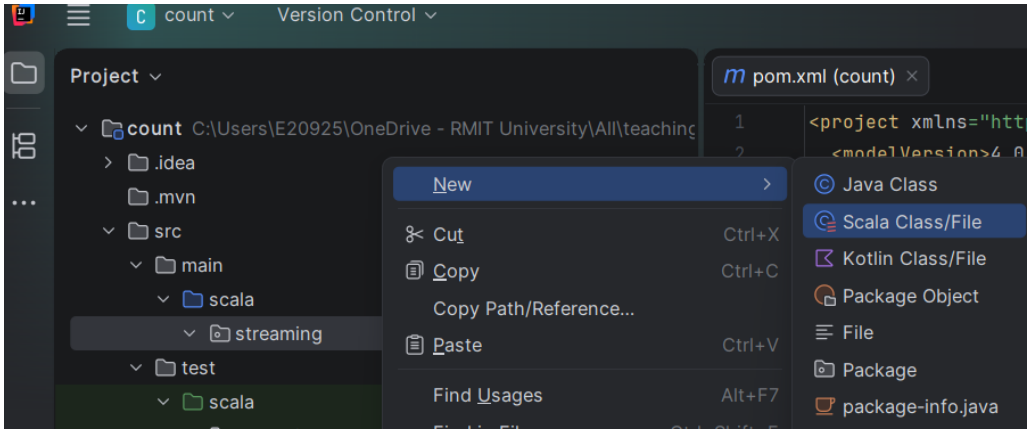
- **main.scala.streaming.App**
- **test.scala.streaming.AppTest**
- **test.scala.streaming.MySpec**

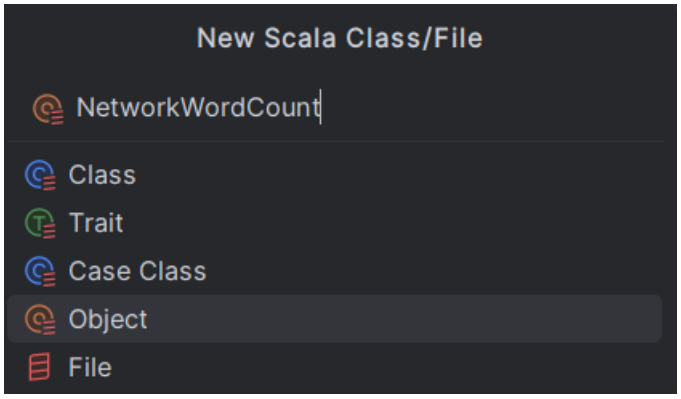






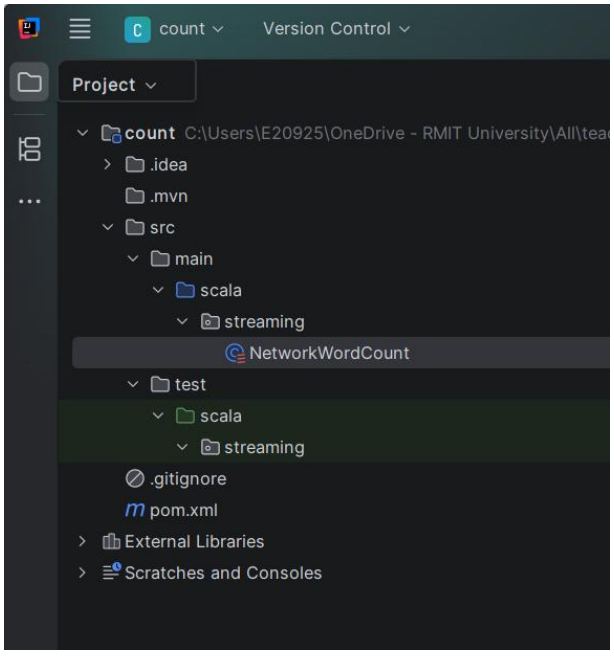
Create a new **scala** object under **main.scala.streaming** package



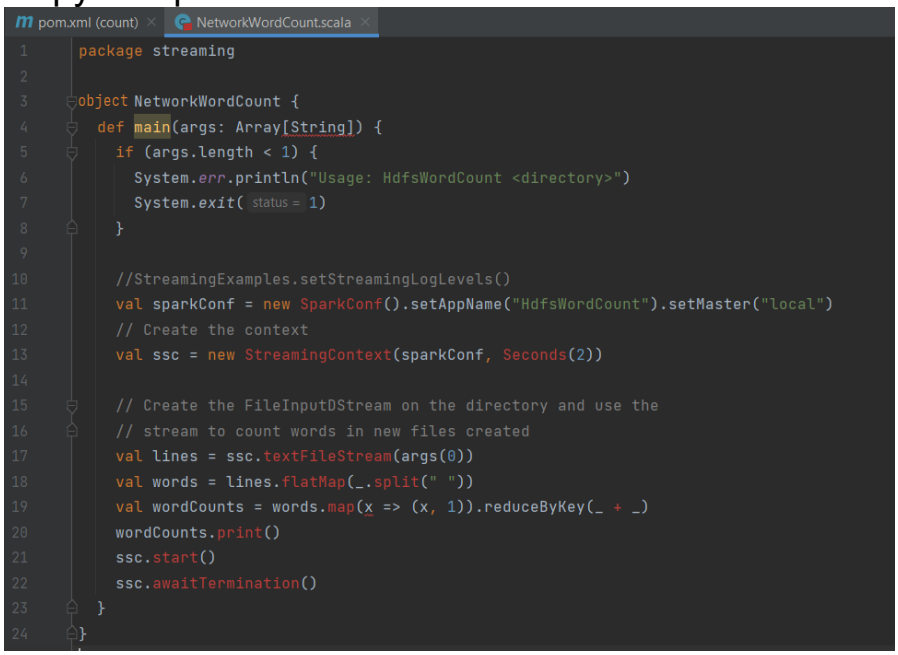


Name the new Scala Object "**NetworkWordCount**", double click **Object**.

A new file "**NetworkWordCount.scala**" will be created



Copy and paste the code in **NetworkWordCount.scala**



double-click **pom.xml**, change

```
<properties>
  <scala.version>2.7.0</scala.version>
</properties>
```

To

```
<properties>
  <scala.version>2.12.18</scala.version>
</properties>
```

Also, add the following dependencies in **pom.xml**

```
<dependency>
  <groupId>org.scala-tools</groupId>
  <artifactId>maven-scala-plugin</artifactId>
  <version>2.15.0</version>
</dependency>

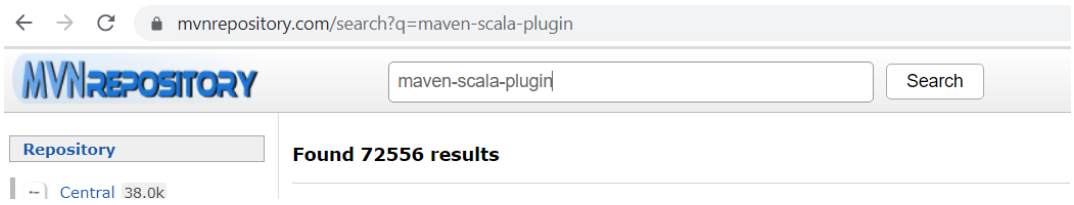
<dependency>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-eclipse-plugin</artifactId>
  <version>2.5.1</version>
</dependency>
```

And add the following dependencies in **pom.xml**

```
<!-- https://mvnrepository.com/artifact/org.apache.spark/spark-core -->
<dependency>
<groupId>org.apache.spark</groupId>
<artifactId>spark-core_2.12</artifactId>
<version>3.1.1</version>
</dependency>
<!-- https://mvnrepository.com/artifact/org.apache.spark/spark-streaming -->
<dependency>
<groupId>org.apache.spark</groupId>
<artifactId>spark-streaming_2.12</artifactId>
<version>3.1.1</version>
<!-- <scope>provided</scope> -->
</dependency>
<!-- https://mvnrepository.com/artifact/org.apache.logging.log4j/log4j-core -->
<dependency>
<groupId>org.apache.logging.log4j</groupId>
<artifactId>log4j-core</artifactId>
<version>2.14.1</version>
</dependency>
```

The following shows an example how dependencies are found from Maven Repository. This is for learning purpose. The five dependencies required for week 11 task have been specified in the last page.

The dependency can be extracted from https://mvnrepository.com/.
For "**maven-scala-plugin**", for example, search for it in the site.



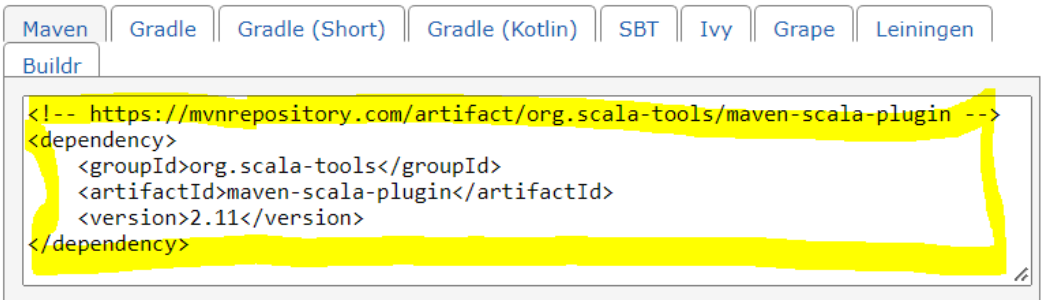
It will return:



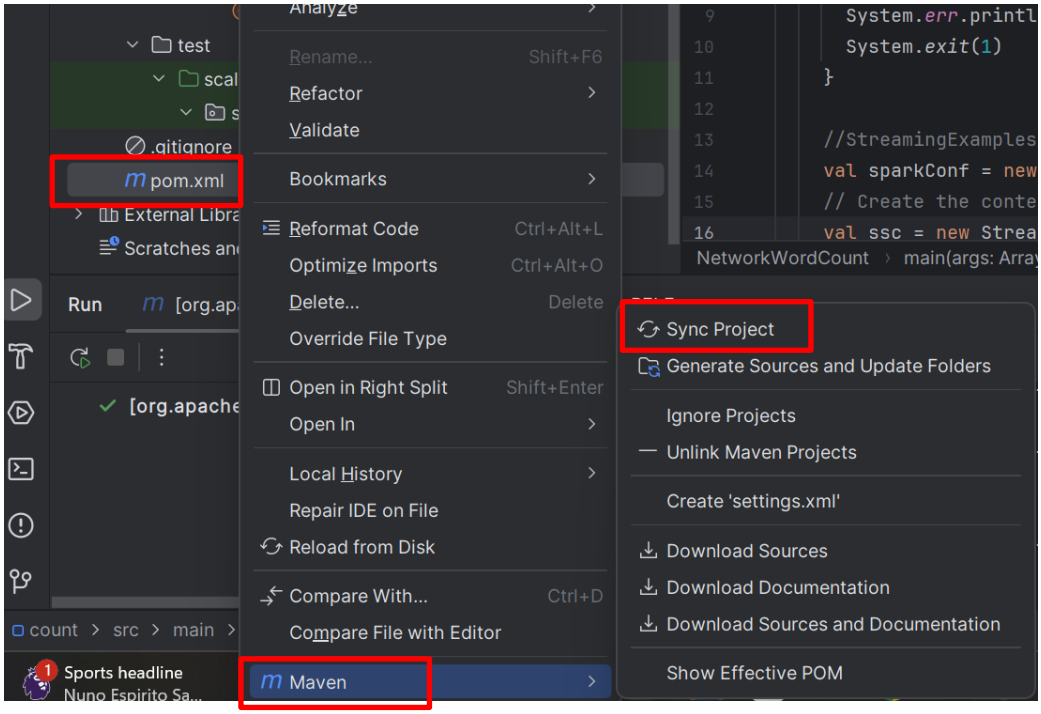
Click "**maven-scala-plugin**", you will see a list where 2.11 is what we need.



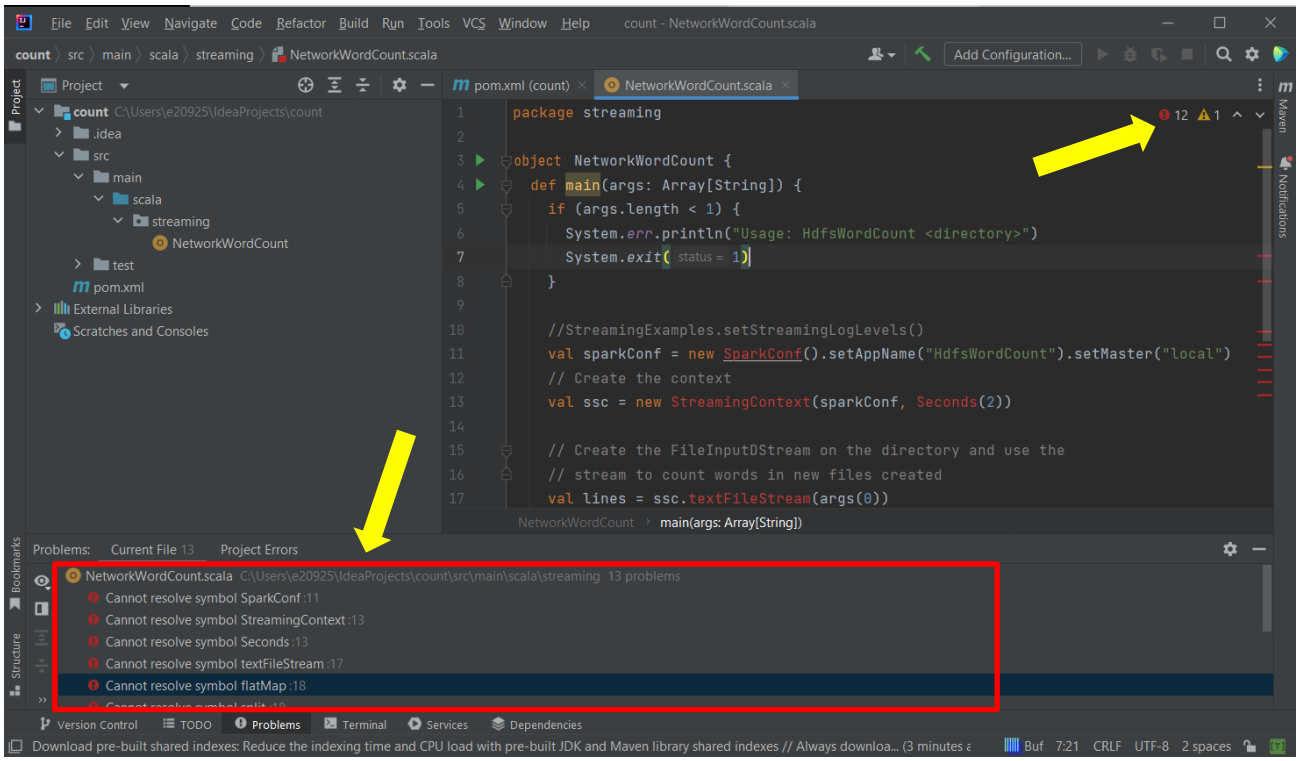
Click 2.11, you will see the following, copy and paste it to **pom.xml**



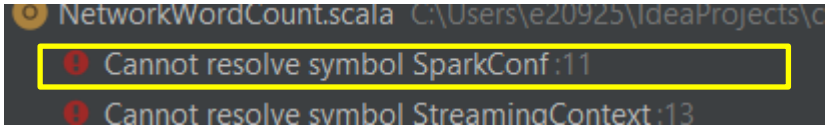
Right click **pom.xml,** select **Maven,** select **Sync Project**



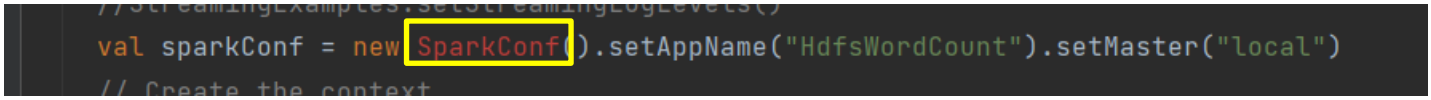
After that, you see error for your code in “**NetworkWordCount.scala**”



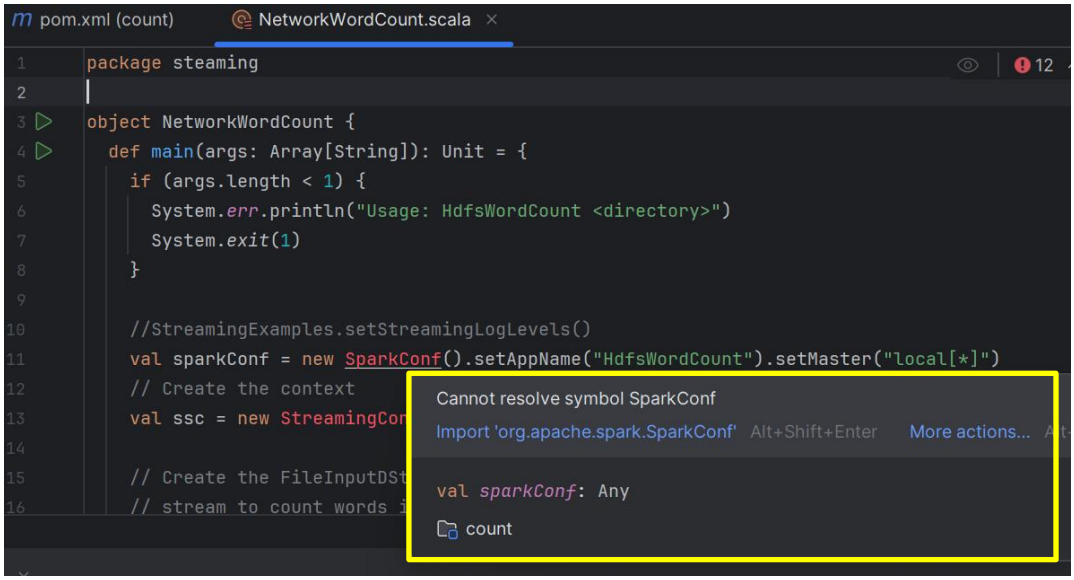
To fix the problem, you need import packages. For the first error,



stop cursor on “**SparkConf**” in **NetworkWordCount.scala**,



a message dialog will pop up.



Click **"Import 'org.apache.sparkSparkConf',** a new import line is added and the error related to SparkConf is fixed (colour changed from red to normal).

For all error message, process in the similar way. **NetworkWordCount.scala** will have **import xxx** as follows:

```
package streaming

import org.apache.spark.SparkConf
import org.apache.spark.streaming.{Seconds, StreamingContext}

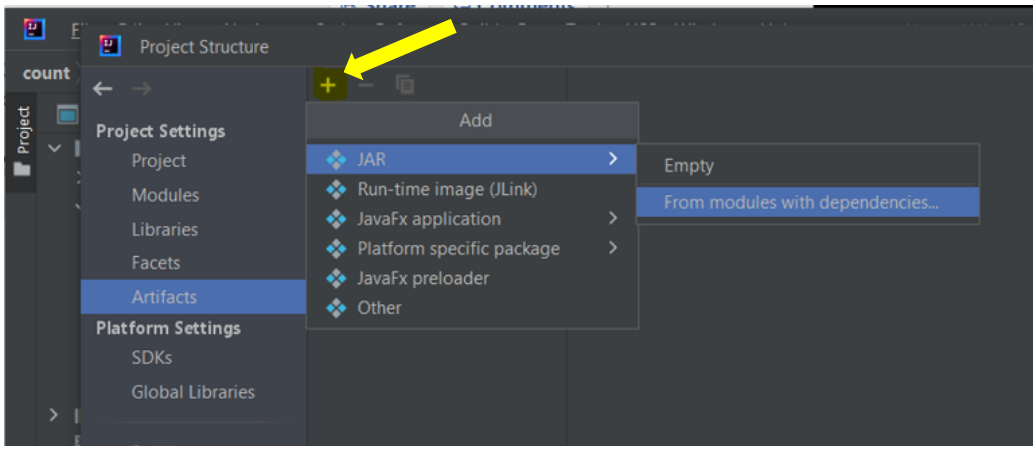
object NetworkWordCount {
  def main(args: Array[String]): Unit = {
    if (args.length < 1) {
      System.err.println("Usage: HdfsWordCount <directory>")
      System.exit( status = 1)
    }

    //StreamingExamples.setStreamingLogLevels()
    val sparkConf = new SparkConf().setAppName("HdfsWordCount").setMaster("local")
    // Create the context
    val ssc = new StreamingContext(sparkConf, Seconds(2))

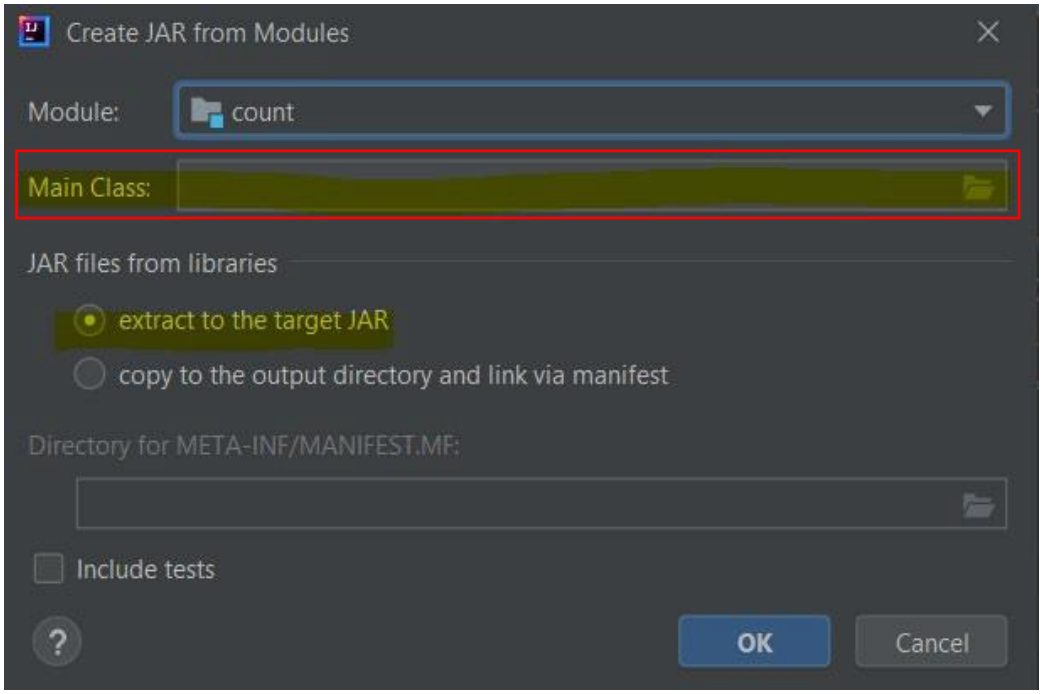
    // Create the FileInputDStream on the directory and use the
    // stream to count words in new files created
    val lines = ssc.textFileStream(args(0))
    val words = lines.flatMap(_.split( regex = " "))
    val wordCounts = words.map(x => (x, 1)).reduceByKey(_ + _)
    wordCounts.print()
    ssc.start()
    ssc.awaitTermination()
  }
}
```

Now, create the .jar file. IntelliJ IDEA enables creation of JAR as an artifact of a project. Do the following steps.

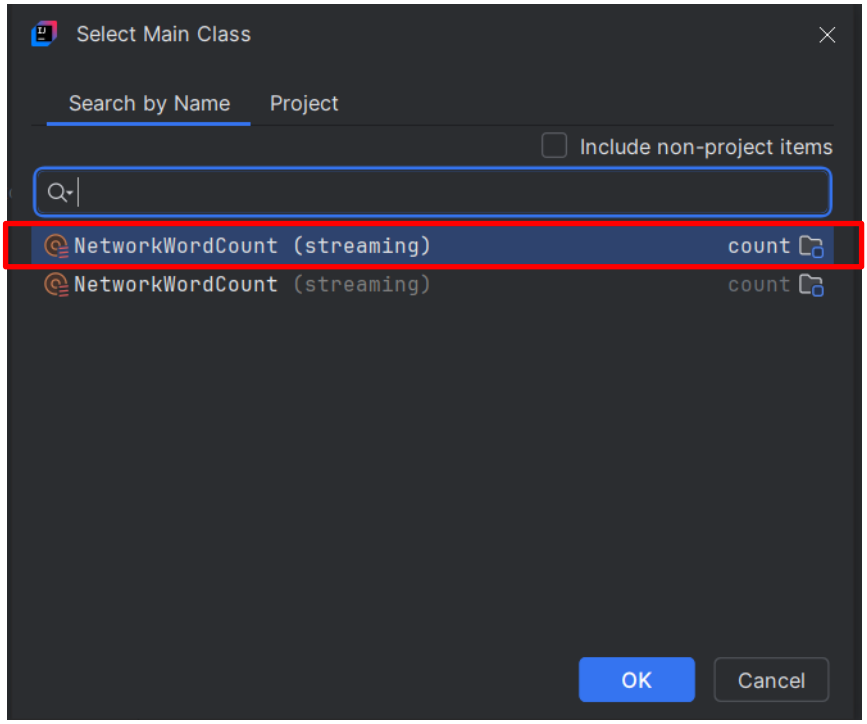
1. From the **File** menu, select **Project Structure...**.
2. From the **Project Structure** window, navigate to **Artifacts** > **the plus symbol + > JAR** > **From modules with dependencies...**.



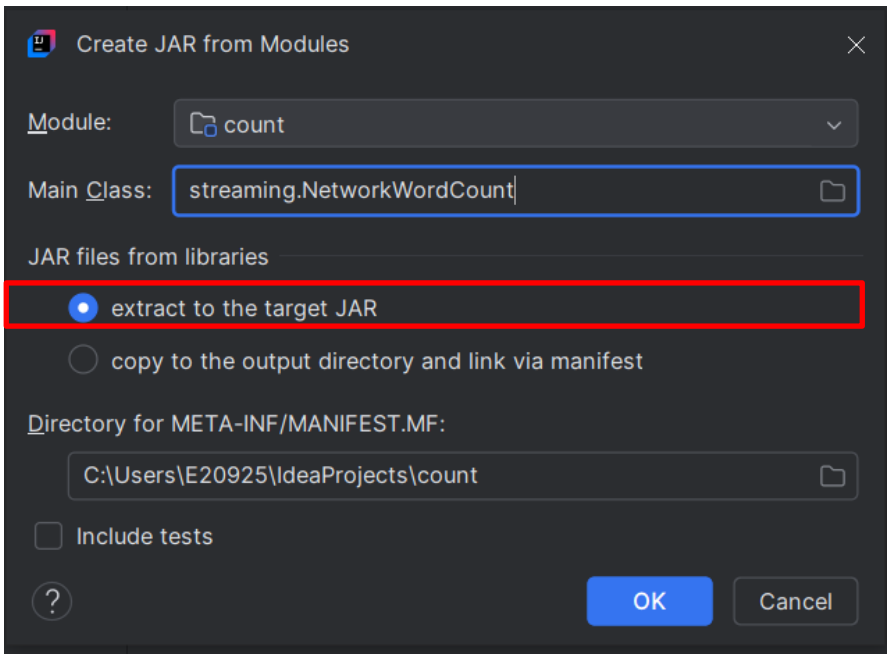
3. In the **Create JAR from Modules** window, select the folder icon in the **Main Class** text box.



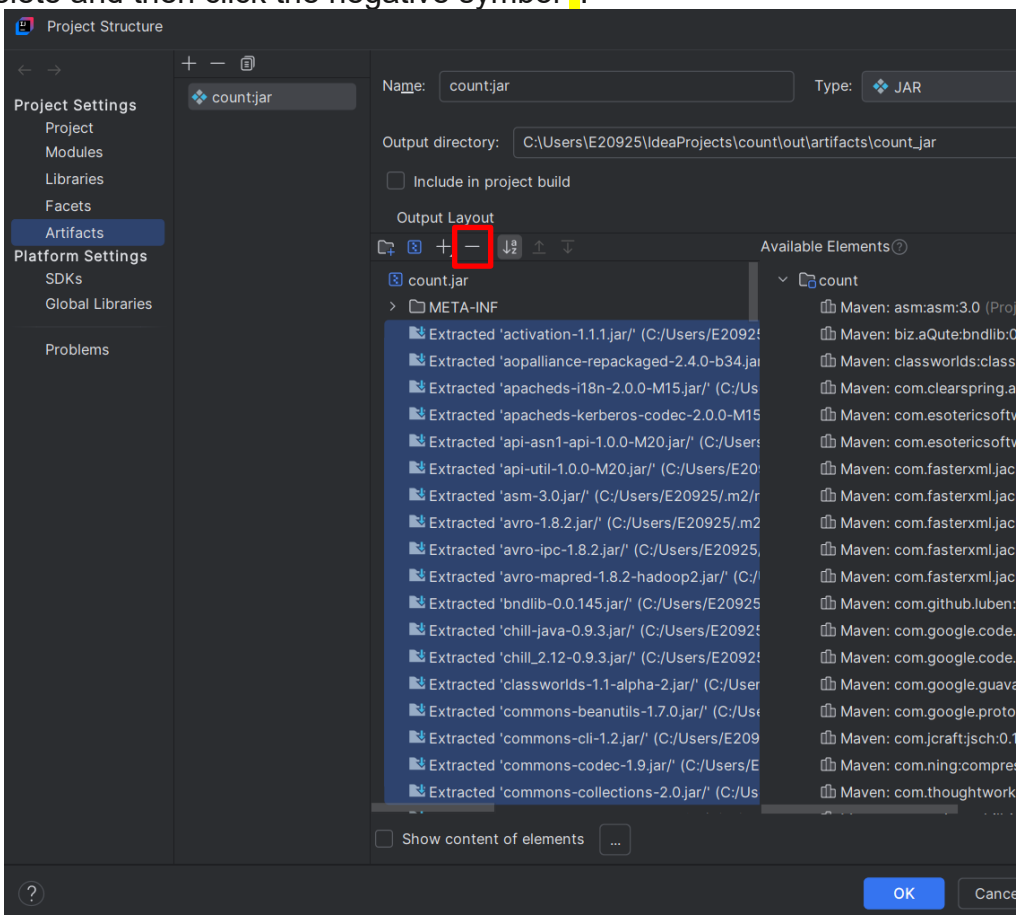
4. In the **Select Main Class** window, select the class that appears by default and then select **OK**.



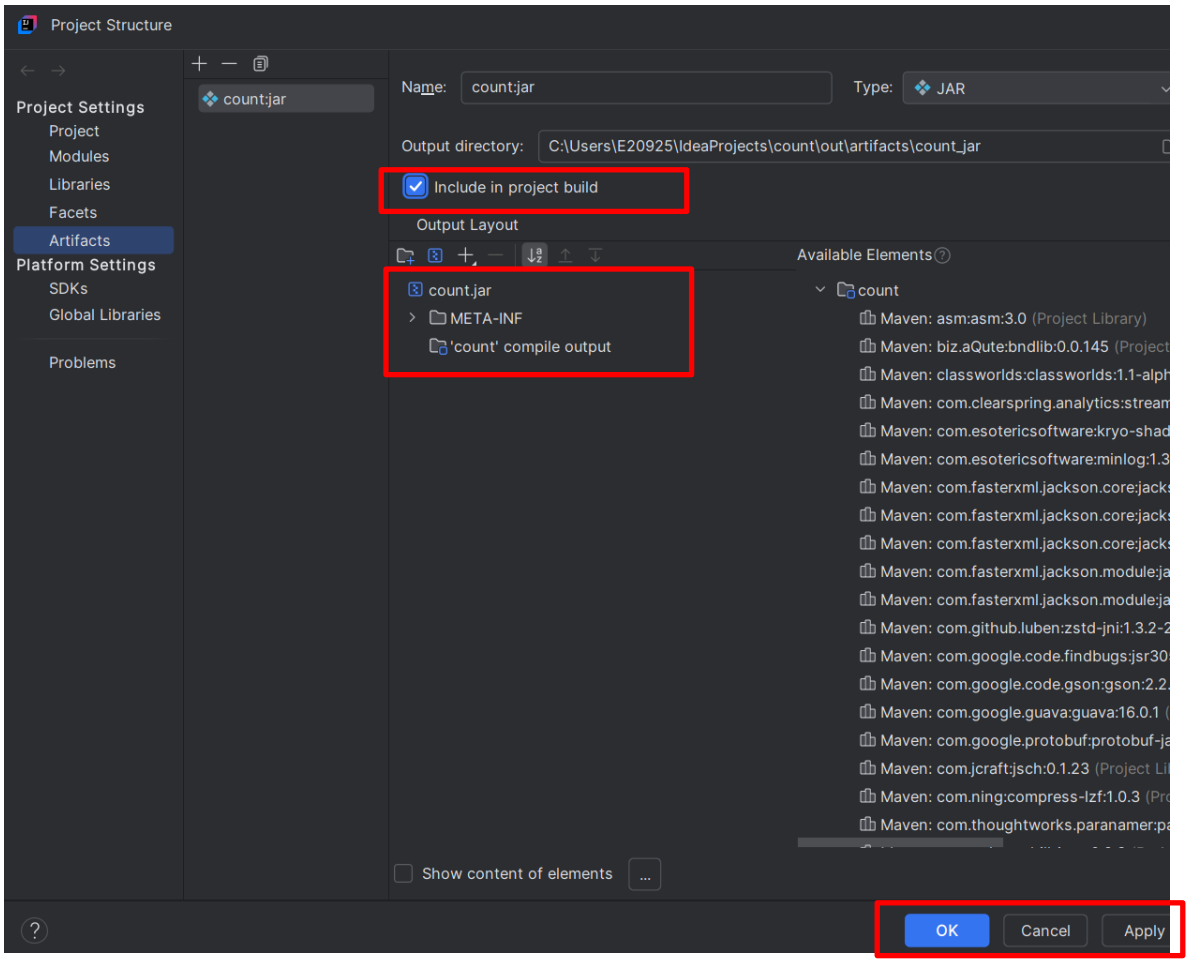
5. In the **Create JAR from Modules** window, ensure the **extract to the target JAR** option is selected, and then select **OK**. Leave **META-INF/MANIFEST.MF** as it is.



6. The **Output Layout** tab lists all the dependencies of the Maven project. For this project, you can remove all dependencies but the last one (‘**count’ compile output**). This will create a small jar (the small jar assumes the running environment will provide all dependencies, not like a deployable jar which include all dependencies). Select the jars to delete and then click the negative symbol -.

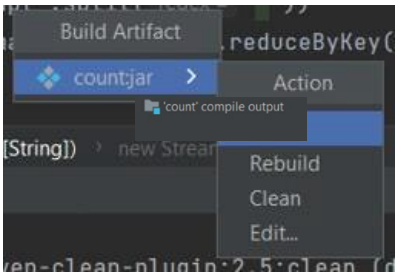


After clicking "**-**", you will see

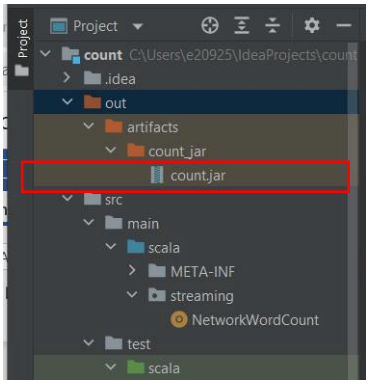


Ensure sure **Include in project build** checkbox is selected. This option ensures that the jar is created every time the project is built or updated. Select **Apply** and then **OK**.

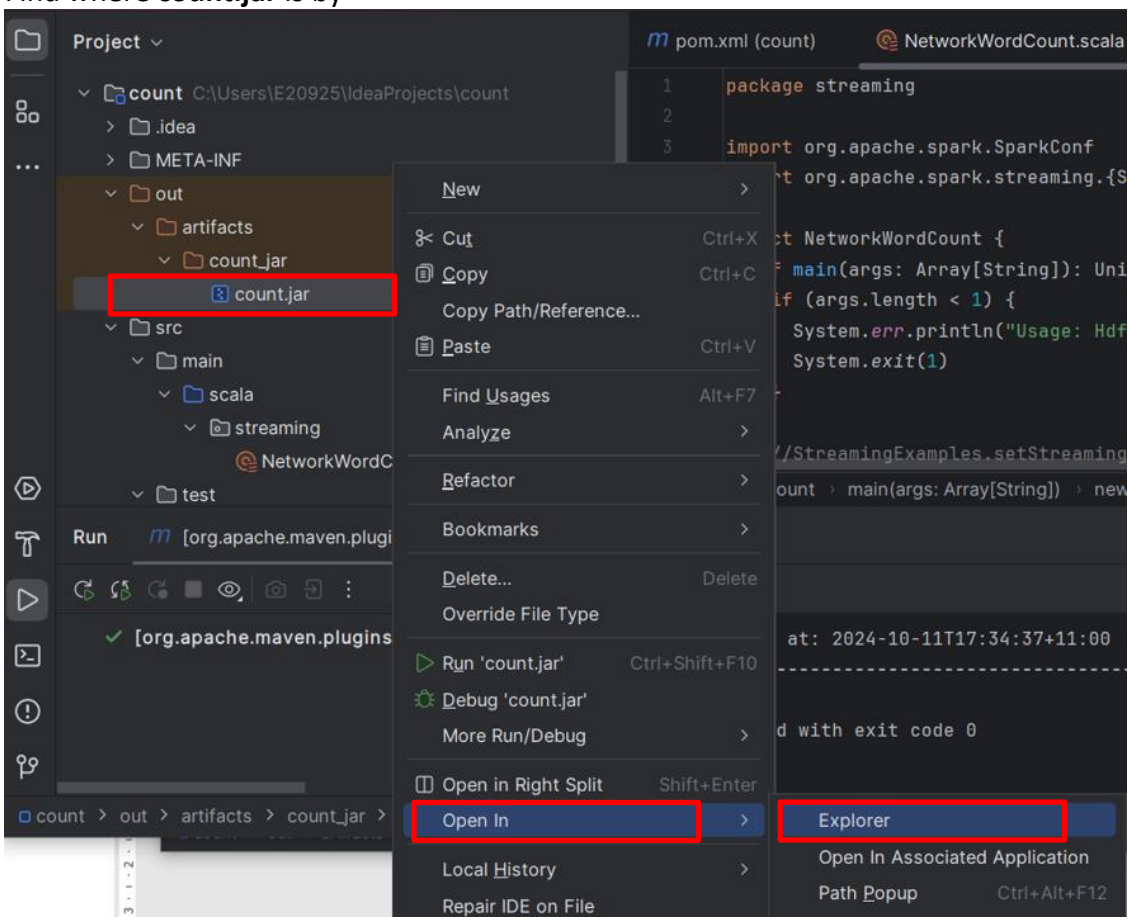
7. To create the jar, navigate to **Build** > **Build Artifacts** > **Build**. The project will be compiled.



The output jar is created under **\out\artifacts\count_jar**.



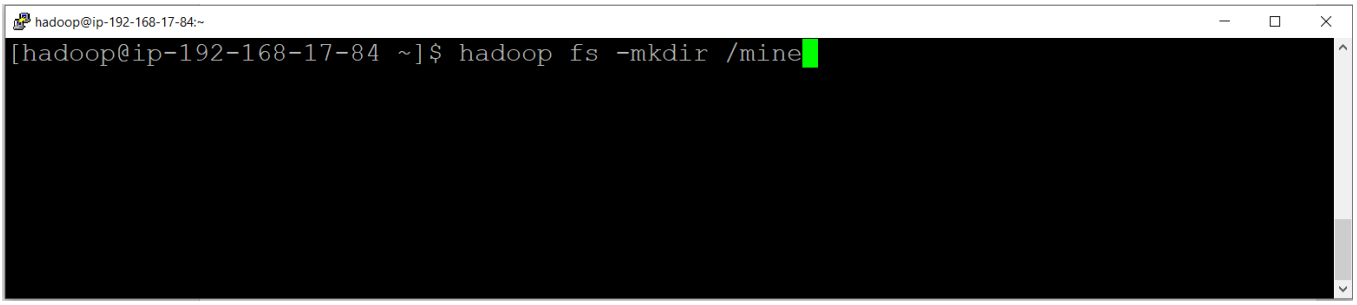
8. Find where **count.jar** is by



9. Upload "**count.jar**" to cluster master node as what learned in previous weeks. Now, you can run **count.jar** to monitor a folder on HDFS, for example, **/mine**.

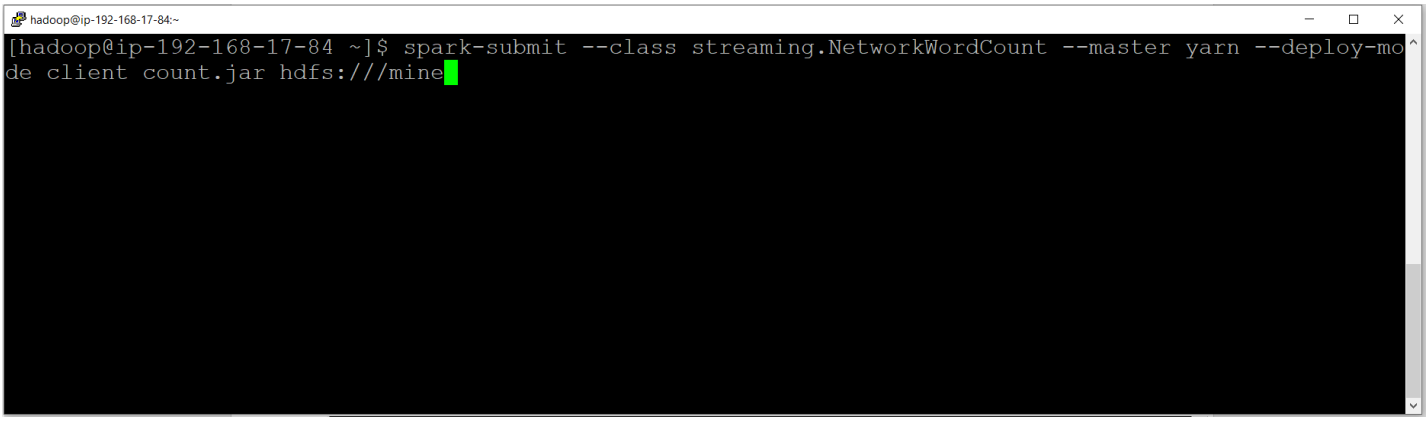
10. Open two terminals.

    The first terminal create folder **/mine** on HDFS

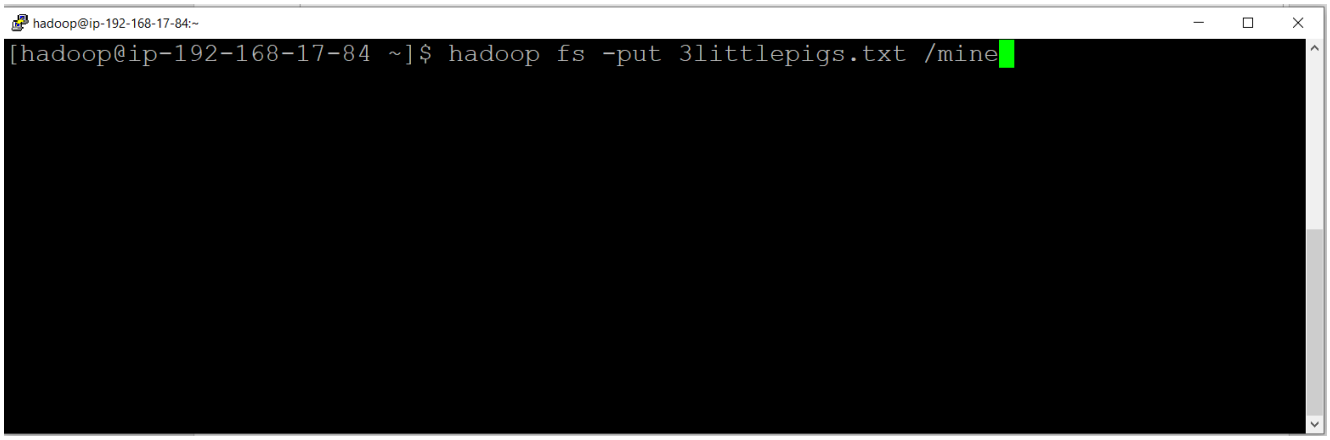


    The second terminal runs **count.jar** to monitor the folder **/mine** on HDFS using command:

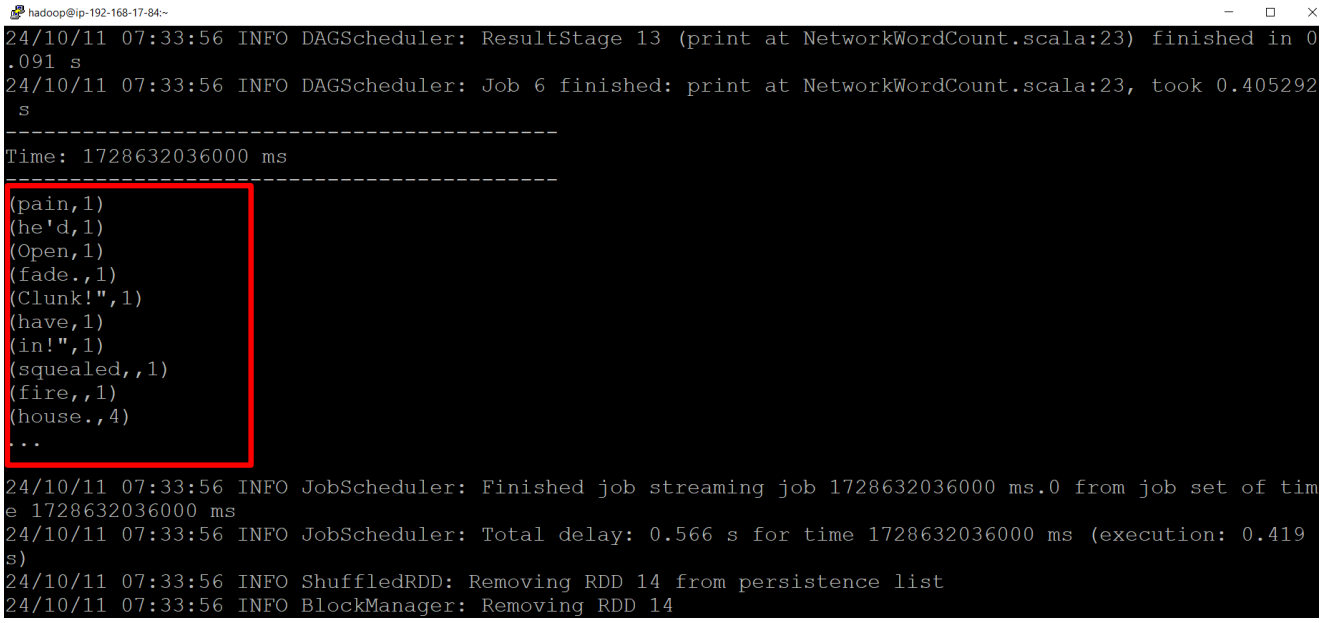
spark-submit --class streaming.NetworkWordCount --master yarn --deploy-mode client count.jar hdfs:///mine



Next, using the first terminal to store a new text file like **3littlepigs.txt** in **/mine**



You will observe the word count in the second terminal like below:



Don’t forget to terminate cluster

$./terminate_cluster.sh